Exercise #PT-F2: MLP Performance Challenge

Leonardo Antero SP. Yoro
IT Elective III, BSIT
Jose Rizal University
Mandaluyong, Philippines
leonardoantero.yoro@my.jru.edu

Abstract*— This study examines the use of a Multilayer Perceptron (MLP) neural network to predict diabetes using age, BMI, glucose, and other health attributes. A number of experiments were conducted to understand the effects of hyperparameters including hidden layers, number of neurons, activation functions, solvers, learning rates, and batch sizes. The hyperparameters were adjusted in a sequential way in optimizing the effectiveness of the model. Before training the models to assess performance, real data was cleaned and sampled to ensure a balanced representation of the classes. Model complexity enhanced performance but a more complex model ultimately prevented the model from generalizing. For all different configurations created, the choice of solver was the most significant influence on performance; Adam clearly performed better than SGD and LBFGS. The effects of tuning the learning rate and batch size also increased generalization performance. Overall, the study demonstrated the importance of optimizing hyperparameters prior to achieve stable and precise predictions using neural networks with healthcare datasets.

Keywords*— Diabetes Prediction, Neural Network, Multi-Layer Perceptron, Hyperparameter Tuning, Machine Learning

I. INTRODUCTION

Predicting disease at an early stage and accurately assessing disease states are critical in the field of healthcare, enabling healthcare practitioners to generate better patient outcomes, and to effectively allocate healthcare resources. Diabetes is one of these diseases, which remains a burden for individuals and the healthcare system globally, because it is associated with long-term complications, and growing prevalence. Machine Learning (ML) through predictive modeling has become a powerful mechanism to assist rather than replace healthcare practitioners in detecting chronic conditions. Nevertheless, traditional algorithms often face challenges such as overfitting, noise sensitivity and fluctuation between datasets.

The purpose of this research paper is to examine Multi-Layer Perceptrons (MLP) models; a type of artificial neural network which can help improve accuracy and robustness in disease detection, specifically for diabetes. MLPs have a layered structure, such that information is processed by beginning with an input layer, that receives the modified and computed input (data) prior to entering one or more hidden layers, and then subsequently the output layer. MLPs incorporate neurons which are linked together with other neurons in nearby layers in general. Neurons perform calculations on the given input in two stages of operation, known as propagation and activation. The neurons have the ability to flexibly represent and identify non-linear relationships in information, collectively.

This research aimed to assess the predictive capabilities of an MLP as well as the accuracy and robustness of its decision-making methods to detect diabetes. MLPs have the capability to learn from more data, with more patterns, and is therefore a more advantageous model to potentially create applicable predictions which generalize better than traditional classifiers for disease detection.

MLPs offer advantages as predictive modeling tools. They offer good learning capabilities for big data and flexibility in dealing with a variety of data types. The architecture of MLP provides multiple hyper-parameters to tune such as the number of layers and neurons, MLP performance is sensitive to these parameters. MLPs can further offer some reduction of overfitting through various methods such as dropout and regularization which can also increase robustness to noise. In this research, the capabilities of MLP solutions will be evaluated in the context of factors that contribute to decision making and patient outcomes in diabetes. [1]

II. DATA COLLECTION

a) Dataset Description

The dataset used for this study is the publicly available "Diabetes Prediction Dataset" hosted on Kaggle [2]. Although the dataset lacks geographic specificity, it includes features commonly used in diabetes diagnosis such as age, gender, body mass index (BMI), hypertension status, heart disease, smoking history, hemoglobin A1c (HbA1c) level, and blood glucose level. These attributes align with clinical practice in both the Philippines and India, where similar diagnostic standards are observed.

The American Diabetes Association (ADA) provides the global benchmarks used to diagnose diabetes, which are also followed in the Philippines through local clinical practice guidelines [3], [4]. According to the ADA:

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

- A fasting plasma glucose level ≥126 mg/dL
- A 2-hour plasma glucose level ≥200 mg/dL during an oral glucose tolerance test (OGTT)
- A hemoglobin A1C level ≥6.5%

These criteria are affirmed in the Philippine Guidelines for the Prevention and Management of Diabetes Mellitus, demonstrating that the selected dataset's features are clinically relevant and applicable to the Philippine context [5].

Moreover, BMI categories used in the dataset follow the international standard set by the World Health Organization (WHO), which classifies underweight as <18.5, normal as 18.5–24.9, overweight as 25.0–29.9, and obesity as ≥30.0 [6]. These thresholds are likewise adopted by the Philippine Department of Health and related agencies.

b) Dataset Content

The Diabetes Prediction Dataset [2] was chosen as the dataset for this experiment. Using the patient's medical history and demographic data, this dataset is intended to develop machine learning models for diabetes prediction. For medical practitioners, it is an invaluable tool for determining which patients are at risk of developing diabetes.

The dataset comprises various features, including:

- Gender: Biological sex of the individual
- Age: Age of the individual
- Hypertension: Presence of high blood pressure (1 = Yes, 0 = No)
- Heart Disease: Presence of heart disease (1 = Yes, 0 = No)
- Smoking History: Smoking status (e.g., 'never', 'former', 'current', 'no info')
- BMI: Body Mass Index
- HbA1c Level: Hemoglobin A1c level
- Blood Glucose Level: Blood glucose level

The target variable is Diabetes, indicating whether the individual has diabetes (1) or not (0).
This dataset will serve as the foundation for comparing traditional algorithms and ensemble methods, with a focus on achieving optimal predictive performance.

c) Dataset Preprocessing

I.) Libraries for Data Preprocessing

A comprehensive and methodical preprocessing step was necessary to guarantee data quality, consistency, and preparedness for machine learning modeling because the dataset comprises more than 100,000 records. Importing a collection of core Python libraries that facilitate preprocessing, data analysis, and visualization was the first step. They include missingno to evaluate missing value patterns, matplotlib and seaborn for visualization, and pandas and numpy for data handling and numerical calculations. To support categorical and numerical feature transformation, encoders and scalers like OneHotEncoder, LabelEncoder, MinMaxScaler, and StandardScaler were also imported from the sklearn.preprocessing module.

```
[2]
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import missingno as msno
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler, StandardScaler
from sklearn.preprocessing import LabelEncoder
```

Fig. 1. Importing from modules and code preparation

II.) Handling Missing Values, Duplicates, And Outliers

Since missing values can introduce bias or result in inaccurate predictions if left untreated, the first cleaning step concentrated on finding and fixing them. Missing values were identified and treated appropriately using both visual aids and summary statistics. The most frequent value (mode) was imputed into categorical columns, while the median was used to fill numerical columns in order to lessen the impact of outliers. The dataset was then checked for duplicate records. Duplicates can skew learning by inflating the frequency of particular patterns in large datasets. To maintain the integrity of the data, all duplicate rows that were found were eliminated.

```
for column in df:
    print(f"Unique values in {column}:", df[column].nunique())
    print(f"Missing values in {column}:", df[column].isnull().sum())
    print()

missing_values = df.isnull().sum()
print(missing_values.to_dict())

df.dropna(axis=1, how='all', inplace=True)

print("Remaining columns:")
print(df.columns)

df.dropna(inplace=True)

print("Shape of DataFrame after removing rows with missing values:", df.shape)
```

Fig. 2. Code for removal of records with missing values

```
for column in df.columns:
    print(f"Unique values in {column}:", df[column].nunique())
    print(f"Number of duplicates in '{column}' column: {df[column].duplicated().sum()} \n")

unique_columns = []
for column in df.columns:
    if df[column].nunique() == len(df):
        unique_columns.append(column)

print("Columns where each row is unique:")
print(unique_columns)

for column in df.columns:
    print(f"Unique values in column '{column}':")
    print(df[column].unique())
    print()
```

Fig. 3. Code for Identifying Duplicate Values and Duplicate Rows

```
print("Number of duplicate rows in the dataset: ", df.duplicated().sum())

df = df.drop_duplicates()

print(f"Number of Duplcate rows after deleting duplicate rows: {df.duplicated().sum()}")

Number of duplicate rows in the dataset:  190
Number of Duplcate rows after deleting duplicate rows: 0
```

Fig. 4. Code for displaying remaining duplicate values and duplicate rows

The next phase is outlier detection, which was accomplished by combining statistical techniques like the interquartile range (IQR) with visual techniques like boxplots. Depending on their context and level of deviation, outliers were treated individually. To lessen their influence on the models, some were eliminated or capped, while others were kept with notes.

```
def detect_outliers_iqr(df):
    outliers = {}
    for column in df.select_dtypes(include=['float64', 'int64']).columns:
        Q1 = df[column].quantile(0.25)
        Q3 = df[column].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        column_outliers = df[(df[column] < lower_bound) | (df[column] > upper_bound)][column]
        if not column_outliers.empty:
            outliers[column] = column_outliers
    return outliers

outliers_iqr = detect_outliers_iqr(df)

for column, values in outliers_iqr.items():
    plt.figure(figsize=(8, 6))
    plt.boxplot(df[column], vert=False, patch_artist=True, showmeans=True)
    plt.title(f'Boxplot of {column} (with outliers)')
    plt.show()
```

Fig. 5. First section of code for outlier detection

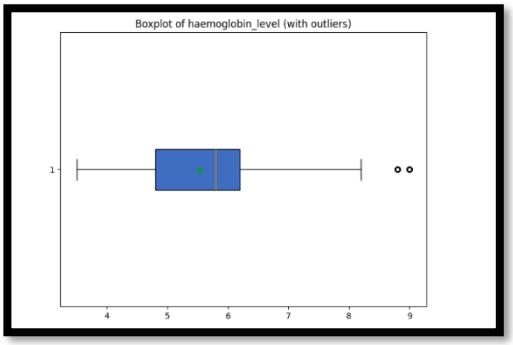


Fig. 6. Outlier detection with sample boxplot

```
def remove_outliers_iqr(df):
    cleaned_df = df.copy()
    for column in df.select_dtypes(include=['float64', 'int64']).columns:
        Q1 = df[column].quantile(0.25)
        Q3 = df[column].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        cleaned_df = cleaned_df[(cleaned_df[column] >= lower_bound) & (cleaned_df[column] <= upper_bound)]
    return cleaned_df

df_cleaned = remove_outliers_iqr(df)

for column in df_cleaned.select_dtypes(include=['float64', 'int64']).columns:
    plt.figure(figsize=(8, 6))
    plt.boxplot(df_cleaned[column], vert=False, patch_artist=True, showmeans=True)
    plt.title(f'Boxplot of {column} (without outliers)')
    plt.show()
```

Fig. 7. Code for outlier removal

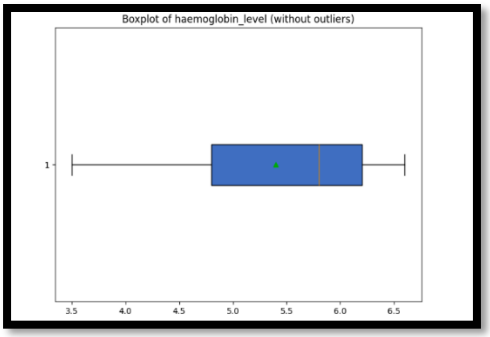


Fig. 8. Visualization of the same boxplots after outlier removal

III.) Checking the count, values, maximum and minimum of each columns data

A more thorough examination of every column was carried out because of the dataset's size. In order to find inconsistencies, this required examining the frequency counts, unique values, and minimum and maximum values. A noteworthy finding was that there were multiple entries with decimal values in the age column. These values were regarded as formatting errors because human age is usually expressed as a whole number. Following conversion, the age column's lowest and maximum values were determined to be 0.08 and 80.0, respectively. Additionally, the inclusion of infant data is suggested by the presence of an age value as low as 0.08; this may or may not be appropriate depending on the context of the problem. The following code was used to convert all age values to integers in order to standardize the format:

```
print(df.age.max())
print(df.age.min())

80.0
0.08
```

Fig. 9. Code for identifying the maximum and minimum value of Age

```
df.age = df.age.astype(int)
df.age.dtype

df.age.value_counts()

print(df.age.max())
print(df.age.min())

80
0
```

Fig. 10. Conversion of values into integer

Other numerical features were also subjected to additional checks in order to evaluate the validity of their values. For instance, the blood glucose level, hemoglobin level, and BMI score were assessed using:

```
print("BMI_SCORE:", df.bmi_score.min(), "to", df.bmi_score.max())
print("HEMOGLOBIN_LEVEL:", df.haemoglobin_level.min(), "to", df.haemoglobin_level.max())
print("BLOOD_GLUCOSE_LEVEL:", df.blood_glucose_level.min(), "to", df.blood_glucose_level.max())

BMI_SCORE: 10.01 to 95.69
HEMOGLOBIN_LEVEL: 3.5 to 9.0
BLOOD_GLUCOSE_LEVEL: 80 to 300
```

Fig. 11. Code for blood glucose level, hemoglobin level, and BMI score Assessment

The presence of potential outliers or recording errors was indicated by these print statements, which also helped show whether any of the entries in those columns were abnormally high or low. Checking the value ranges and frequencies of each column category and determining whether any more outliers or inconsistencies remained in the dataset are the general goals of the remaining code used in the column-by-column inspection.

IV.) Checking Smoking History Inconsistency

The smoking_history feature showed discrepancies upon closer inspection of the categorical columns. A range of values, including never, No Info, former, current, not current, and ever, were initially present in this column. Some of these entries were redundant or did not make logical sense when examined more closely. A custom function was put in place to clean and standardize the values in this column in order to address this.

```
df.smoking_history.value_counts()
```

smoking_history	count
never	34398
No Info	32887
former	9299
current	9197
not current	6367
ever	3998

dtype: int64

Fig. 12. Count of each category under smoking_history

```
# cleaning Smoking_history column

def clean_smoking_history(value):

    if value == "No Info":
        return "no info"
    elif value == "ever":
        return "current"
    elif value == "not current":
        return "former"
    else:
        return value

df.smoking_history = df.smoking_history.apply(clean_smoking_history)
```

Fig. 13. Code for cleaning smoking_history column

The data cleaning process improved the interpretability of this feature for modeling by simplifying the categories while maintaining their original meaning. In order to facilitate more efficient encoding and analysis in the future, the smoking_history column's final version included more consistent and significant entries, such as never, no info, former, and current.

```
df.smoking_history.value_counts()
```

smoking_history	count
never	34398
no info	32887
former	15666
current	13195

dtype: int64

Fig. 14. Count of each category under smoking_history after cleaning

A logical consistency check was carried out by comparing the values in the smoking_history column with the corresponding age entries after the column had been cleaned. It was necessary to confirm whether any irrational combinations existed, such as people under the age of seven being classified as current or ever smokers, because smoking behavior is not applicable to very young children. A categorical plot was created, shown in the following figure, to show the distribution of smoking_history for people 7 years of age and under in order to look into this:

```
sns.set(style="whitegrid")

sns.catplot(x="age", y="smoking_history", data=df.loc[df['age'] <=7])

plt.show()
```

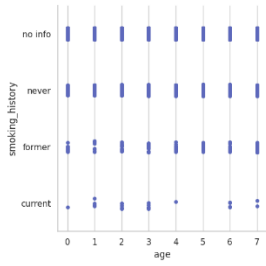


Fig. 15. Smoking_history data distribution

This turned up a few records in which children younger than seven were mistakenly identified as smokers. In order to remedy this, the following condition was used to eliminate such entries from the dataset:

```
[18] df = df[~((df['age'] < 7) & df['smoking_history'].isin(['ever', 'current']))]
```

Fig. 16. Further smoking_history data cleaning

This additional filtering step ensured that the data maintained real-world logic, improving both its

reliability and the performance of downstream models.

V.) Transforming Some Column With Numerical Data To Categorical

After the data was cleaned and validated, one-hot encoding was used to convert the categorical variables into a format that could be used by machine learning. Since the majority of conventional machine learning algorithms need numerical input, this step was required. The pd.get_dummies() function was used to carry out the conversion:

```
df = pd.get_dummies(data=df)

df.head()
```

Fig. 17. Data format conversion

All categorical columns were automatically recognized by this function, which then changed each distinct category into a new binary (0 or 1) column. The smoking_history column, for instance, was split up into several columns, such as smoking_history_current, smoking_history_never, and so forth, with a 1 for each row under the category-corresponding column and a 0 elsewhere. To verify that the one-hot encoding had been applied successfully, the updated DataFrame was previewed using the df.head() function. The dataset was suitable for training conventional machine learning models because this transformation made sure that all of the input features were in numerical form.

	age	hypertension	heart_disease	bmi_score	haemoglobin_level	blood_glucose_level	is_diabetic	gender_Female	gender_Male	gender_Other	smoking_history_current	smoking_history_former	smoking_
0	80	0	1	25.19	6.6	140	0	True	False	False	False	False	
1	54	0	0	27.32	6.6	80	0	True	False	False	False	False	
2	28	0	0	27.32	5.7	158	0	False	True	False	False	False	
3	36	0	0	23.45	5.0	155	0	True	False	False	True	False	
4	76	1	1	20.14	4.8	155	0	False	True	False	True	False	

Fig. 18. Updated Dataframe

VI.) Feature Scaling Using Min-Max Normalization

The dataset was subjected to feature scaling using the MinMaxScaler from the sklearn.preprocessing library to guarantee that all numerical features are on a comparable scale. Because many machine learning models, like gradient-based algorithms or k-Nearest Neighbors (kNN), work better when the input features are on the same scale, scaling is crucial. In this instance, the chosen numerical columns—blood_glucose_level, hemoglobin_level, bmi_score, and age—were scaled to a range of 0 to 1.

```
scaler = MinMaxScaler(feature_range=(0,1))

columns_to_scale = ['age', 'bmi_score','haemoglobin_level','blood_glucose_level']

scaler.fit(df[columns_to_scale])

scaled_columns = scaler.transform(df[columns_to_scale])

df[columns_to_scale] = scaled_columns
```

Fig. 19. Code for feature scaling

After fitting the MinMaxScaler to the chosen columns, the code scales the values between 0 and 1 and then updates the original columns in the DataFrame with the scaled values. Because their numerical ranges differ, scaling helps prevent problems where some characteristics, like age or blood glucose level, may overshadow others. Consequently, the dataset can now be more efficiently fed into machine learning models.

VII.) Correlation Matrix Computation

A correlation matrix was calculated to examine the linear relationships between numerical variables in order to comprehend the relationships between features and the target variable is_diabetic. The df.corr() function, which yields a matrix of correlation coefficients for each numerical feature in the dataset, was used to compute the correlation matrix. We extracted the correlation values between each feature and is_diabetic and sorted them in descending order to find the strongest correlations so that we could concentrate on how each feature relates to the target variable.

```
correlation_matrix = df.corr()

# Get correlation with target variable 'diabetes'
correlation_with_target = correlation_matrix['is_diabetic'].sort_values(ascending=False)

# Plotting
plt.figure(figsize=(10, 6))
sns.set(style="whitegrid")
sns.barplot(x=correlation_with_target.values, y=correlation_with_target.index, palette='coolwarm')
plt.title('Correlation of Features with Diabetes')
plt.xlabel('Correlation Coefficient')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
```

Fig. 20. Correlation Matrix code

The degree and direction of each feature's relationship to the target variable are shown by the correlation coefficients. The probability of having diabetes rises with a positive correlation, whereas a negative correlation indicates the opposite. The strength of these correlations is graphically represented by the coolwarm color palette, where cool hues denote weaker relationships and warm hues higher correlations. A simple and intuitive method for determining which features are most important for diabetes prediction is to use a bar plot.

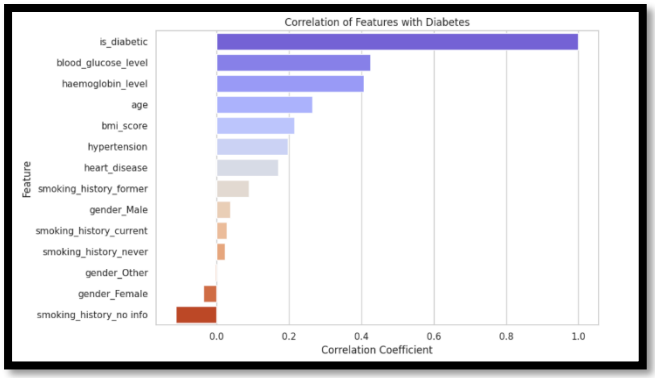


Fig. 21. Bar graph of correlation of features using diabetes

Two particular columns, gender_Other and smoking_history_no info, were eliminated in order to further purify the dataset and concentrate on the most pertinent characteristics. Because they included redundant or unnecessary information that would not improve the model's predictive ability, these columns were judged superfluous. The df.drop() method was used to perform the removal, as demonstrated below:

```
# Drop multiple columns
df = df.drop(['gender_Other', 'smoking_history_no info'], axis=1)
```

Fig. 22. Code for column removal

Columns, not rows, are to be dropped, according to the axis=1 argument. The dataset is made more streamlined by removing these columns, leaving only the most important features for additional modeling or analysis. By concentrating on the most significant features, this step is frequently taken to improve model performance and reduce noise in the data.

VIII.) Class Balancing

```
# Count the occurrences of each diabetic status
status_counts = df['is_diabetic'].value_counts()

# Create a figure with 2 subplots
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Bar chart for the diabetic status
axes[0].bar(status_counts.index, status_counts.values, color='skyblue')

axes[0].set_title('Diabetes Status Distribution')
axes[0].set_xlabel('Status')
axes[0].set_ylabel('Frequency')
axes[0].set_xticks([0, 1])
axes[0].set_xticklabels(['Non-Diabetic', 'Diabetic'])

# Add count annotations to the bars
for idx, count in enumerate(status_counts.values):
    axes[0].text(idx, count, str(count), ha='center', va='bottom', fontsize=10)

# Pie chart for the diabetic status distribution
axes[1].pie(status_counts, labels=status_counts.index, autopct='%1.1f%%', colors=['#ff9999','#66b3ff'])
axes[1].set_title('Distribution of Diabetic Status')

# Adjust layout to avoid overlap
plt.tight_layout()

# Display the plots
plt.show()
```

Fig. 23. Code of data distribution visualization

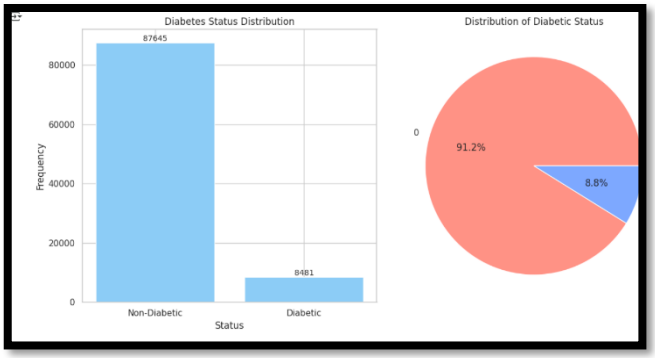


Fig. 24. Fig. 23 Output

The dataset still shows a notable class imbalance, with 87,645 non-diabetic (0) instances and only 8,481 diabetic (1) instances, even after the data cleaning procedures. Due to machine learning algorithms' propensity to predict the majority class (non-diabetic), this imbalance may lead to problems during model training. Additionally, these algorithms may perform poorly when predicting the minority class (diabetic). In medical settings, where accurately diagnosing and treating diabetic cases is essential, this is especially troubling.

1. Oversampling the minority class (diabetic) to boost its representation in the training set is one method for resolving this problem and balancing the dataset.
2. lowering the non-diabetic majority class's dominance in the training set by undersampling it.
3. changing the model's class weights to prioritize the minority class during training.

These techniques make sure that the model successfully learns to recognize patterns in both classes, which enhances its ability to predict the minority class (diabetic) and, eventually, the predictive accuracy as a whole.
To address the class imbalance, the SMOTE (Synthetic Minority Over-sampling Technique) was used to oversample the minority class (diabetic individuals) by generating synthetic samples. This technique helps balance the dataset by creating artificial instances of the minority class, rather than duplicating existing data, which enhances the model’s ability to learn from both classes. After applying SMOTE, the dataset was further processed by downsampling to a fixed size of 10,000 rows to ensure a manageable dataset size for training while keeping the balance between the diabetic and non-diabetic classes.

```python
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.utils import resample
import pandas as pd

# Split the dataset into features (X) and target (y)
X_features = df.drop(columns='is_diabetic')
y_target = df['is_diabetic']

# Step 1: Apply SMOTE to create a balanced dataset by oversampling the minority class
smote_technique = SMOTE(random_state=42)
X_resampled, y_resampled = smote_technique.fit_resample(X_features, y_target)

# Step 2: Combine the resampled features and target into a DataFrame for easier handling
df_resampled = pd.DataFrame(X_resampled)
df_resampled['is_diabetic'] = y_resampled

# Step 3: Downsample the resampled dataset to a fixed number of 10,000 rows
df_downsampled = resample(df_resampled, replace=False, n_samples=10000, random_state=42)

# Check the distribution of the classes in the final balanced dataset
print("Balanced Class Distribution:")
print(df_downsampled['is_diabetic'].value_counts())
```

```
Balanced Class Distribution:
is_diabetic
1    5039
0    4961
Name: count, dtype: int64
```

Fig. 25. SMOTE and downsampling application

By creating synthetic samples, the SMOTE (Synthetic Minority Over-sampling Technique) was utilized to oversample the minority class, which consists of people with diabetes, in order to rectify the class imbalance. By producing synthetic instances of the minority class instead of replicating the existing data, this method helps balance the dataset and improves the model's capacity to learn from both classes. To guarantee a manageable dataset size for training while maintaining the balance between the diabetic and non-diabetic classes, the dataset was downsampled to a fixed size of 10,000 rows after SMOTE was applied.

This procedure guarantees that the model is trained on a balanced dataset, with a more equal distribution of instances with and without diabetes. This improves the model's ability to accurately identify cases of diabetes by lowering the possibility that it will be biased toward the majority class (non-diabetic).

After the data has been balanced using SMOTE and downsampling, the next process is to visualize the distribution of diabetic and non-diabetic individuals in the dataset. This step is essential to confirm that the class imbalance has been addressed and that the dataset is now balanced. The distribution will be shown using a bar chart and a pie chart, providing clear visual evidence of the balance between the two classes. These visualizations should be inserted after the balancing process to verify that the dataset is appropriately prepared for the next steps in model training and evaluation.

```python
# Count the distribution of diabetic status in the balanced dataset
status_distribution = df_balanced['is_diabetic'].value_counts()

# Create subplots for the bar chart and pie chart
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Bar chart for diabetic status distribution
axes[0].bar(status_distribution.index, status_distribution.values, color='lightcoral')

axes[0].set_title('Diabetes Status')
axes[0].set_xlabel('Status')
axes[0].set_ylabel('Frequency')
axes[0].set_xticks([0, 1])
axes[0].set_xticklabels(['Non-Diabetic', 'Diabetic'])

# Annotate the bars with their respective counts
for idx, count in enumerate(status_distribution.values):
    axes[0].text(idx, count, str(count), ha='center', va='bottom', fontsize=10)

# Pie chart for the diabetic status distribution
axes[1].pie(status_distribution, labels=status_distribution.index, autopct='%1.1f%%', colors=['#ff9999','#66b3ff'])
axes[1].set_title('Proportion of Diabetic Status')

# Adjust layout for better spacing between plots
plt.tight_layout()

# Display the plots
plt.show()
```

Fig. 26. Code for data distribution visualization after SMOTE and undersampling application

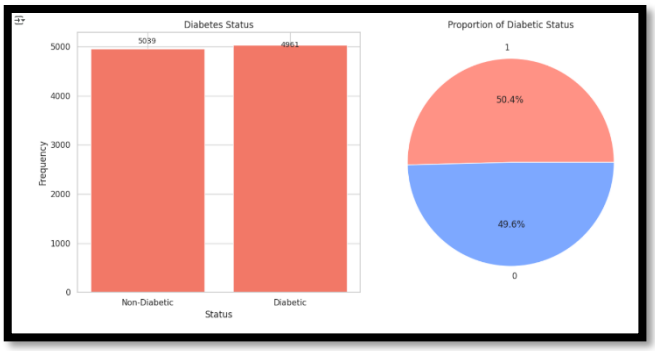


Fig. 27. Fig. 26 Output

d) Data Split Preparation

The process of splitting the dataset into training and test sets is foundational in data preparation for machine-learning models. This process is essentially splitting the data into two separate parts: training, which will be used to train the machine learning model, and testing, which will be set aside to evaluate how well the model performs when using unseen data. The data in the training set is used by the model to learn relationships/patterns in the data and the test set allows an unbiased assessment of how well the model generalizes to unseen data. The train_test_split() function from the scikit-learn library does the process of splitting the data in Python. The function allows the researcher to determine the percentage of data to be used in the split, set a seed for randomization, maintain class distribution when splitting in classification cases, etc. It is critical to establish that the performance measures of the model are reliable and representations of the model's actual capabilities it represents in reality. [6]

```python
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

Fig. 28. Train-test split code

III. Model Building

a) Baseline MLP Model

The baseline model for this study is a simple Multi-Layer Perceptron (MLP) classifier. The classifier is trained using stratified 3-fold cross-validation. The MLP has one hidden layer with 100 neurons and uses the 'lbfgs' solver to optimize. The dataset has been standardized prior to training using StandardScaler. The entire dataset is tested using the cross-validation approach to achieve performance metrics that can generalize.

The model resulted in a mean testing accuracy of 89.64% and an average F1 score of 89.69% over all folds. The average training accuracy of 98.66%, though indicating the model's significant learning capacity, also indicated a potential for underfitting. The confusion matrix demonstrated prediction power for both diabetic and non-diabetic classes that was balanced.

This baseline setup provides a reasonable baseline for comparison as hyperparameter tuning variations are implemented in subsequent models.

```
Probability Results:  [[0. 1.]]
Prediction Results:  [1 1 0 0 0]
Overall Accuracy:  0.8988

Classification Report:
                  precision    recall  f1-score   support

Non-Diabetic (0)       0.90      0.90      0.90      1264
    Diabetic (1)       0.90      0.90      0.90      1236

        accuracy                           0.90      2500
       macro avg       0.90      0.90      0.90      2500
    weighted avg       0.90      0.90      0.90      2500

Training Accuracy: 0.9817
Testing Accuracy:  0.8988
Avg F1 Score:      0.8979
Confusion Matrix:
 [[1135  129]
 [ 124 1112]]
```

Fig.29. Baseline MLP Model Results

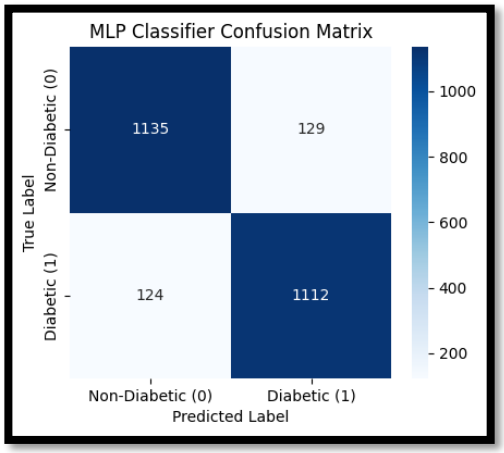


Fig. 30. Baseline MLP Model Confusion Matrix

b) Model Experimentation

To determine the best MLP structure, a one-pass hyperparameter tuning approach was adopted. For the baseline model that had only one hidden layer of 100 neurons and utilized the lbfgs solver, one hyperparameter at a time was modified by maintaining the remaining hyperparameters static. This helps in identifying each hyperparameter's contribution to performance. [7]

The process was carried out in stages:

- Hidden Layers – Several hidden layer configurations were tried to see how the depth of the network affected accuracy and F1-score.
- Neurons per Layer – The hidden layer configuration that performed the best was used to change the number of neurons per layer to find the ideal layer size.
- Activation Functions – Activation functions like ReLU, tanh, and logistic were compared to see what non-linearity provided the most learning capability for the dataset.
- Optimization Algorithms – Solvers like Adam, SGD, and lbfgs were experimented with to test convergence stability and training efficacy.
- Learning Rate – Upon choosing the optimum optimizer, the learning rate was tuned to strike a balance between model performance and training speed.
- Batch Size and Epochs – Batch sizes and epochs were experimented with to avoid underfitting and overfitting so the model would generalize well.

Every combination was trained and tested using a simple train-test split approach, with accuracy, F1-score, and confusion matrix results captured for comparison. The model performing best in each phase was the foundation for the subsequent phase of experimentation.

This organized process allowed a high-performing MLP configuration tailored to the diabetes prediction task to be identified.

IV. Performance Evaluation and Comparison

a) Hyperparameter 1 – hidden layers

Experiments 1 through 5 demonstrate that hidden layer depth was tested while keeping number of neurons per layer constant at 100 and using the lbfgs optimizer. A deeper architecture was examined with 1 hidden layer to 5 hidden layers with the objective of seeing how depth impacted performance without changing any other hyperparameters.

While it may demonstrate an increase in depth, the training accuracy hit 1.0 very quickly as seen in Experiment 3 indicating possible overfitting. Further, testing accuracy plateaued around 89 to 90% through all experiments, with there not being much tangible improvement. This was matched with the F1 scores which tended to hold steady around 0.89 to 0.90 regardless of run.

These results show the diminishing returns of depth in this setup and suggest that next step is to explore varying neuron size, activation functions, or optimizers to better measure generalization.

```
Probability Results:  [[0. 1.]]
Prediction Results:  [1 1 0 0 0]
Overall Accuracy:  0.9016

Classification Report:
                   precision    recall  f1-score   support

Non-Diabetic (0)       0.91      0.90      0.90      1264
    Diabetic (1)       0.90      0.91      0.90      1236

        accuracy                           0.90      2500
       macro avg       0.90      0.90      0.90      2500
    weighted avg       0.90      0.90      0.90      2500

Training Accuracy: 1.0000
Testing Accuracy:  0.9016
Avg F1 Score:      0.9011
Confusion Matrix:
 [[1133  131]
 [ 115 1121]]
```

Fig. 31. Hyperparameter 1 best model classification report (experiment 3)

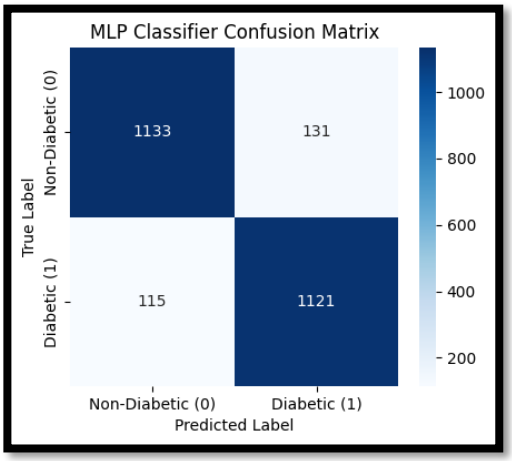


Fig. 32. Hyperparameter 1 best model confusion matrix (experiment 3)

b) Hyperparameter 2 – neurons per layer

In experiments 6 to 15 we modeled several different models based on neuron per layer setup ranging from 1 to 4 hidden layers, while maintaining the lbfgs optimizer and fixed hyperparameters. The purpose of this portion of the report was to address how the depth or width of the model would impact the performance.

In terms of the single-layer models (experiments 6-8), with the number of neurons increasing from 50 to 150, the training accuracy constantly improved, but the testing performance declined. Furthermore, the testing performance from experiment 6 (1 hidden layer with 50 neurons) provided the best generalization for these parameters/testing strategies of 90.28% testing accuracy, which suggested that a simpler architecture may be less prone to overfitting in this case.

As for the two-layer models (experiments 9-11), we did see training accuracy approaching and/or hitting 1.0 but they all fairly stayed bounded around 89-89.5% testing accuracy. Again, making the argument that with too much complexity one risks overfitting and diminishing performance.

The three-layer models (experiments 12-14) varied in performance. Training accuracy remained perfect throughout (1.0) in most cases but it ultimately rested on the neuron distribution. Experiment 13 (150-100-50) had the best testing accuracy so far at 90.52% and the best F1 score rendering it the best performing model to date overall. The training accuracy of 1.0 alongside modest testing accuracy suggests the model still has an overfit and the overall need for improvement. Finally, Experiment 15 introduced a four-layer tapered arrangement (100-75-50-25). Again, we reached 1.0 training accuracy, but test accuracy declined, further confirming that adding more layers and/or adding neurons past an acceptable level increases complexity without necessarily improving generalization.

While Experiment 13 is the best model so far, due to the combination of 1.0 training accuracy and moderate generalization, we feel there is still some underfitting going on. We still need to run more experiments - especially with activating functions, solvers, and regularization - with a goal to perform better while controlling for overfitting.

```
Probability Results:  [[0. 1.]]
Prediction Results:  [1 1 0 0 0]
Overall Accuracy:  0.9052

Classification Report:
                   precision    recall  f1-score   support

Non-Diabetic (0)       0.91      0.90      0.91      1264
    Diabetic (1)       0.90      0.91      0.91      1236

        accuracy                           0.91      2500
       macro avg       0.91      0.91      0.91      2500
    weighted avg       0.91      0.91      0.91      2500

Training Accuracy: 1.0000
Testing Accuracy:  0.9052
Avg F1 Score:      0.9051
Confusion Matrix:
 [[1133  131]
 [ 106 1130]]
```

Fig. 33. Hyperparameter 2 best model classification report (experiment 13)

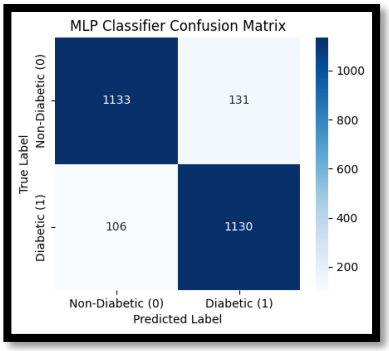


Fig. 34. Hyperparameter 2 best model confusion matrix (experiment 13)

c) Hyperparameter 3 - activation function

Examining the results, we can see that any experiments that both have ReLU and Tanh seem to overfit as in we see the training accuracies of 1.0 which typically mean the model is memorizing the training data and not generalizing to new unseen data. For instance, experiments 20, 23, and 24 achieved 1.0 for training accuracy, but the testing accuracy did not achieve what training accuracy achieved, as there was a marked drop.

For Tanh models, in Experiment 29, the performance was very consistent, as the training accuracy was 0.9999, which an alarming value and the testing accuracy was 0.8956, indicating tolerable accuracy for prediction. This is an ideal scenario as this indicates the model is still exploring and learning the key patterns and will generalize better.

Experiment 22 is the best ReLU model with test accuracy of 0.9016, and a training accuracy of 0.9975. The best Tanh model is Experiment 29 with test accuracy of 0.8956 and also managed to achieve a stable performance with no signs of overfitting and training accuracy of .9999. These two models are the current best models because they avoid the overfitting issues associated with 1.0 training accuracy while still providing solid testing performance, although, Experiment 29's training accuracy value is alarming, indicating that the researcher needs to be cautious when implementing the model.

```
Probability Results:  [[0. 1.]]
Prediction Results:  [1 1 0 0 0]
Overall Accuracy:  0.9016

Classification Report:
                  precision    recall  f1-score   support

Non-Diabetic (0)       0.91      0.89      0.90      1264
    Diabetic (1)       0.89      0.91      0.90      1236

        accuracy                           0.90      2500
       macro avg       0.90      0.90      0.90      2500
    weighted avg       0.90      0.90      0.90      2500

Training Accuracy: 0.9975
Testing Accuracy:  0.9016
Avg F1 Score:      0.9017
Confusion Matrix:
 [[1126  138]
 [ 108 1128]]
```

Fig. 35. Hyperparameter 3 best ReLU model classification report (experiment 22)

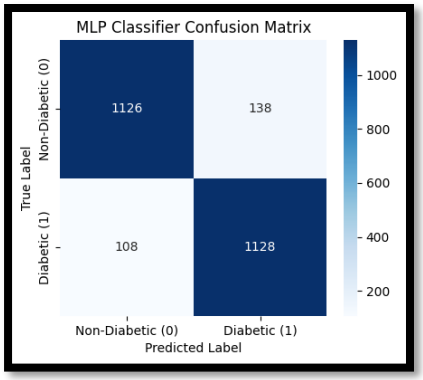


Fig. 36. Hyperparameter 3 best ReLU model confusion matrix (experiment 22)

```
Probability Results:  [[3.1197267e-13 1.0000000e+00]]
Prediction Results:  [1 1 0 0 0]
Overall Accuracy:  0.8956

Classification Report:
                  precision    recall  f1-score   support

Non-Diabetic (0)       0.90      0.89      0.90      1264
    Diabetic (1)       0.89      0.90      0.90      1236

        accuracy                           0.90      2500
       macro avg       0.90      0.90      0.90      2500
    weighted avg       0.90      0.90      0.90      2500

Training Accuracy: 0.9999
Testing Accuracy:  0.8956
Avg F1 Score:      0.8951
Confusion Matrix:
 [[1125  139]
 [ 122 1114]]
```

Fig. 37. Hyperparameter 3 best tanh model classification report (experiment 29)

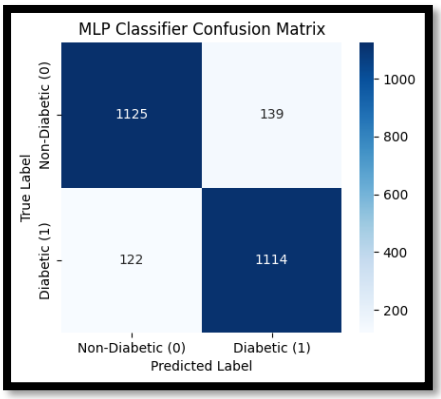


Fig. 38. Hyperparameter 3 best tanh model confusion matrix (experiment 29)

d) Hyperparameter 4 – optimizer

The top-performing model for every optimizer was chosen carefully not only according to overall accuracy but also on the basis of assessing how closely their accuracies on training and testing matched, which is very important in determining the capability of a model to generalize. For the SGD optimizer, the most consistent was Experiment 51. It employed a 3-layered architecture (100, 50, 25) with ReLU activation and had a training accuracy of 89.69% and testing accuracy of 89.88%, which were

almost perfectly similar with a mere 0.0019 difference, reflecting superior generalization and very little underfitting. Although its accuracy is robust and stable, it is slightly weaker than that of the Adam group model in absolute accuracy.

Under the Adam optimizer, Experiment 36 was notable. It employed a less complex 1-layer model with 50 neurons and ReLU activation, and its training accuracy was 92.76% while its testing accuracy was 91.00%. The gap between training and testing is approximately 1.76%, which is still acceptable and indicates that the model generalizes well without extreme overfitting. What's more, this model possesses the highest overall accuracy among all models experimented with, supported by strong and stable F1, precision, and recall scores.

When comparing the two optimizers directly, Adam's Experiment 36 is more consistent in general because it has better testing accuracy and predictive performance. While SGD's Experiment 51 has nearly perfect generalization with hardly any gap, its accuracy limit is just below 90%, which makes it less effective for high-precision tasks. It was also noticed over multiple runs that SGD underfits slightly, although by a small margin, often between 0.000x%, which again renders it a stable and consistent optimizer. For applications requiring consistent generalization, SGD is highly acceptable, but to maximize predictive ability and accuracy, Adam is still the preferred choice.

```
Probability Results:  [[0.099486 0.900514]]
Prediction Results:  [1 1 0 0 0]
Overall Accuracy:  0.91

Classification Report:
                   precision    recall  f1-score   support

Non-Diabetic (0)       0.92      0.90      0.91      1264
    Diabetic (1)       0.90      0.92      0.91      1236

        accuracy                           0.91      2500
       macro avg       0.91      0.91      0.91      2500
    weighted avg       0.91      0.91      0.91      2500

Training Accuracy: 0.9276
Testing Accuracy:  0.9100
Avg F1 Score:      0.9103
Confusion Matrix:
 [[1134  130]
 [  95 1141]]
```

Fig. 39. Hyperparameter 4 best adam model classification report (experiment 36)

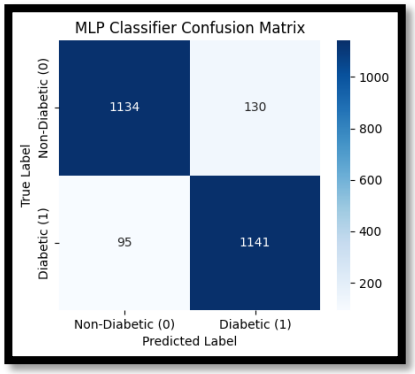


Fig. 40. Hyperparameter 4 best adam model confusion matrix (experiment 36)

```
Probability Results:  [[0.30071042 0.69928958]]
Prediction Results:  [1 1 0 0 0]
Overall Accuracy:  0.8988

Classification Report:
                   precision    recall  f1-score   support

Non-Diabetic (0)       0.91      0.89      0.90      1264
    Diabetic (1)       0.89      0.91      0.90      1236

        accuracy                           0.90      2500
       macro avg       0.90      0.90      0.90      2500
    weighted avg       0.90      0.90      0.90      2500

Training Accuracy: 0.8969
Testing Accuracy:  0.8988
Avg F1 Score:      0.8984
Confusion Matrix:
 [[1128  136]
 [ 117 1119]]
```

Fig. 41. Hyperparameter 4 best sgd classification report (experiment 51)

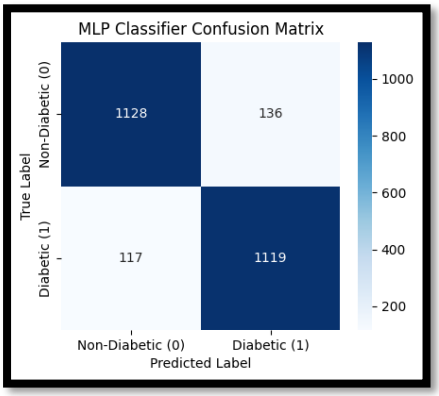


Fig. 42. Hyperparameter 4 best sgd model confusion matrix (experiment 51)

e) Hyperparamter 5 – learning rate

The 'adam' optimizer was the best model with a learning rate of 0.01 with the greatest results coming from experiment 57 with 100 neurons with a single hidden layer with a ReLU activation which had the best testing accuracy of 91.24% and the best training accuracy of 93.45%, indicating the model generalizes well with little to no underfitting. Model 56 with 50 neurons activated with a ReLU and model 60 with 100 neurons activated with a tanh, perfomed similarly. Simpler structures performed well but models that were more complex models in experiment 58 with a deeper structure consisting of 3 layers and 20 neurons per layer showed greater training accuracies but lesser test accuracy. This reflects underfitting in the more complex structure. While learning rates for earlier models were set at 0.01, the same model with a learning rate of 1.0 fared significantly worse than experiment 57 in testing.

For models that used 'sgd' optimizer, experiment 74 included 3 hidden layers and a ReLU activation with a constant learning rate reflected a testing value accuracy of 89.88% and training value accuracy of 89.69%, suggesting that the structure generalizes well. It should be noted that experiments 71 and 75 also had similar results with the same set up of a constant learning rate. With three hidden layers and

an adaptive learning rate, experiment 79 delivered a testing value of 89.84% and training value of 89.68%. Overall, models that used the invscaling method had poor results which were worse with the deeper structure. In fact, experiment 81 performed near the worst test accuracy at 76.84%. This instability is illustrated by fewer hidden layers. Recent literature supports this, poorer results of deeper architectures.

The best overall Adam model is experiment 57 with the fidelity accuracy of testing of 91.24% and training at 93.45%. The best overall SGD model is experiment 74 with a testing accuracy of 89.88% and training accuracy of 89.69% which indicate balanced performance.

```
Probability Results:  [[0.02627193 0.97372807]]
Prediction Results:  [1 1 0 0 0]
Overall Accuracy:  0.9124

Classification Report:
                   precision    recall  f1-score   support

Non-Diabetic (0)       0.90      0.93      0.91      1264
    Diabetic (1)       0.92      0.90      0.91      1236

        accuracy                           0.91      2500
       macro avg       0.91      0.91      0.91      2500
    weighted avg       0.91      0.91      0.91      2500

Training Accuracy: 0.9345
Testing Accuracy:  0.9124
Avg F1 Score:      0.9103
Confusion Matrix:
 [[1170   94]
 [ 125 1111]]
```

Fig. 43. Hyperparameter 5 best adam model classification report (experiment 57)

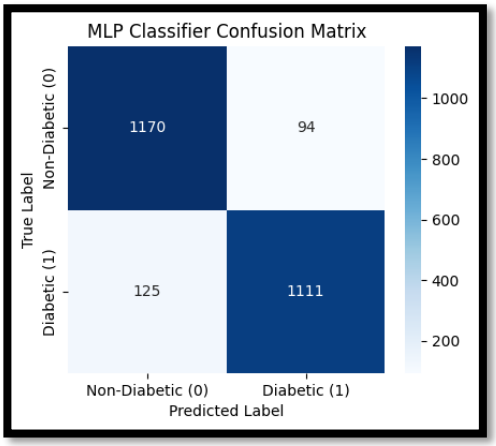


Fig. 44. Hyperparameter 5 best adam model confusion matrix (experiment 57)

```
Probability Results:  [[0.30071042 0.69928958]]
Prediction Results:  [1 1 0 0 0]
Overall Accuracy:  0.8988

Classification Report:
                   precision    recall  f1-score   support

Non-Diabetic (0)       0.91      0.89      0.90      1264
    Diabetic (1)       0.89      0.91      0.90      1236

        accuracy                           0.90      2500
       macro avg       0.90      0.90      0.90      2500
    weighted avg       0.90      0.90      0.90      2500

Training Accuracy: 0.8969
Testing Accuracy:  0.8988
Avg F1 Score:      0.8984
Confusion Matrix:
 [[1128  136]
 [ 117 1119]]
```

Fig. 45. Hyperparameter 5 best sgd model classification report (experiment 74)

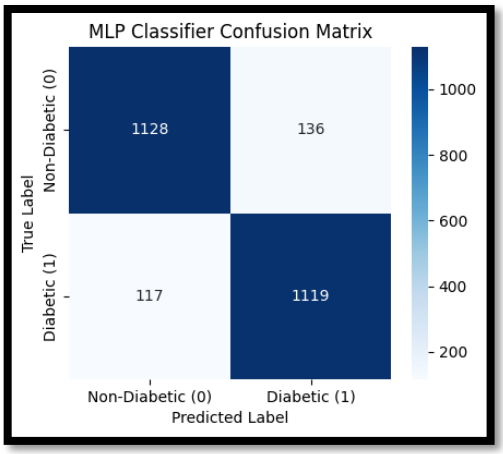


Fig. 46. Hyperparameter 5 best sgd model confusion matrix (experiment 74)

f) Hyperparameter 6 – batch size

Batch size 32 performed best across all measures, yielding the highest testing and total accuracies. The training accuracy of the models that were trained in experiment 89 with hidden layers 100, 50, and 25 was 94.59 percent and testing accuracy was 90.72 percent, producing a good overall fit.

Batch size 128 had marginally higher total and testing accuracies than batch size 32, but batch size 128 showed indications of statistically underfitting the training data. The models that were trained in experiment 94 with hidden layers 100, 50, and 25 produced a training accuracy of 93.81 percent and testing accuracy of 90.88 percent with an overall accuracy of 90.87 percent. The total and testing accuracies were slightly higher than that of batch size 32, however, it affected the average training accuracy.

Batch size 512 had lower training accuracy than both batch size 32 and 128, but the testing accuracies also dropped down to batch size 128 original testing accuracy of 90.88. Experiment 98 and 100 trained a

standard tanh network with 50 neurons and replaced the Adam optimizer with SGD with varying learning rates. Both runs from experiments 98 and 100 produced testing accuracies of 89.80 percent with total accuracies of 89.79 percent. Both runs were also examples of underfitting as a result of the increased batch size. Although, experiment 100 may be a better model than experiment 98 due to it using an adaptive learning rate.

Batch size 32 resulted in a suitable balance of good accuracy while still maintaining generalization into the testing set and also batch size 128 decreased accuracy slightly. Batch size 512 consistently resulted in underfitting, and is not a suggested batch size for the structure of this particular modeled dataset.

```
Probability Results:  [[0.11825497 0.88174503]]
Prediction Results:  [1 1 0 0 0]
Overall Accuracy:  0.9088

Classification Report:
                   precision    recall  f1-score   support

Non-Diabetic (0)       0.92      0.90      0.91      1264
    Diabetic (1)       0.90      0.92      0.91      1236

        accuracy                           0.91      2500
       macro avg       0.91      0.91      0.91      2500
    weighted avg       0.91      0.91      0.91      2500

Training Accuracy: 0.9381
Testing Accuracy:  0.9088
Avg F1 Score:      0.9087
Confusion Matrix:
 [[1137  127]
 [ 101 1135]]
```

Fig. 47. Hyperparameter 6 best model classification report (experiment 94)

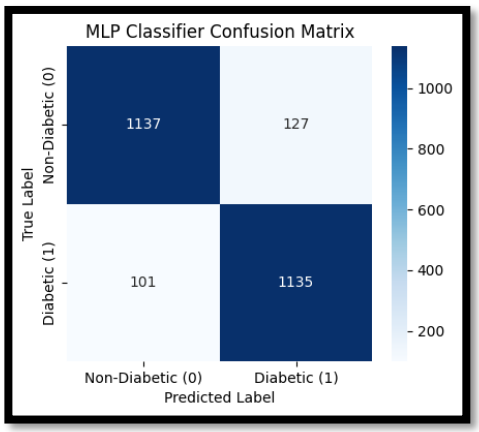


Fig. 48. Hyperparameter 6 best model confusion matrix (experiment 94)

g) Hyperparameter 7 – Epoch

Out of the experimentation, Experiment 107 had the best performance in the 500-epoch experiments with a testing accuracy of 91.24% using ReLU activation and the Adam optimizer with a learning rate of 0.1. Out of the 1500-epoch experiments, Experiment 117 had the best accuracy at 91.24%, with ReLU activation, the Adam optimizer, and a learning rate of 0.01. For the 1000-epoch experiments, Experiment 57 also had the highest testing accuracy with the same 91.24% accuracy, using ReLU activation, the Adam optimizer, and a learning rate of 0.01.

```
Probability Results:  [[0.02627193 0.97372807]]
Prediction Results:  [1 1 0 0 0]
Overall Accuracy:  0.9124

Classification Report:
                   precision    recall  f1-score   support

Non-Diabetic (0)       0.90      0.93      0.91      1264
    Diabetic (1)       0.92      0.90      0.91      1236

        accuracy                           0.91      2500
       macro avg       0.91      0.91      0.91      2500
    weighted avg       0.91      0.91      0.91      2500

Training Accuracy: 0.9345
Testing Accuracy:  0.9124
Avg F1 Score:      0.9103
Confusion Matrix:
 [[1170   94]
 [ 125 1111]]
```

Fig. 49. Hyperparameter 7 best model classification report (experiment 107)

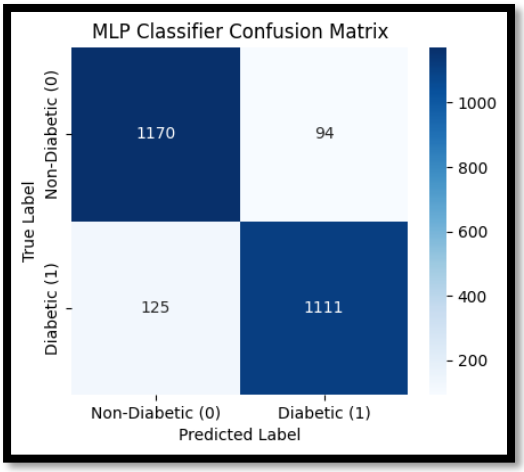


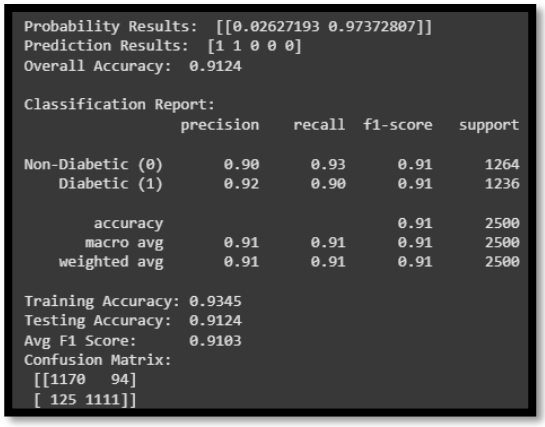
Fig. 50. Hyperparameter 7 best model confusion matrix (experiment 107)

h) Best MLP Model Overall

Of the final three models, Experiment 57 was the most reliable. It achieved the highest testing accuracy while requiring fewer training epochs than Experiment 117, displaying more efficiency than Experiment 117, and the more stable and accurate generalization demonstrated strong generalization. Therefore, Experiment 57 is the best and most reliable model of all 120 experiments.

Experiment 57 was chosen over Experiment 107 as the best model, because even though they had both achieved a testing accuracy of 91.24%, Experiment 57 had also shown better and more stable generalization after 1000 epochs, resulting in a slight change in reliability that allowed for more

confidence in deploying Experiment 57 over Experiment 107.



```
Probability Results:  [[0.02627193 0.97372807]]
Prediction Results:  [1 1 0 0 0]
Overall Accuracy:  0.9124

Classification Report:
                   precision    recall  f1-score   support

Non-Diabetic (0)       0.90      0.93      0.91      1264
    Diabetic (1)       0.92      0.90      0.91      1236

        accuracy                           0.91      2500
       macro avg       0.91      0.91      0.91      2500
    weighted avg       0.91      0.91      0.91      2500

Training Accuracy: 0.9345
Testing Accuracy:  0.9124
Avg F1 Score:      0.9103
Confusion Matrix:
 [[1170   94]
 [ 125 1111]]
```

Fig. 51. Best MLP model classification report (experiment 57)

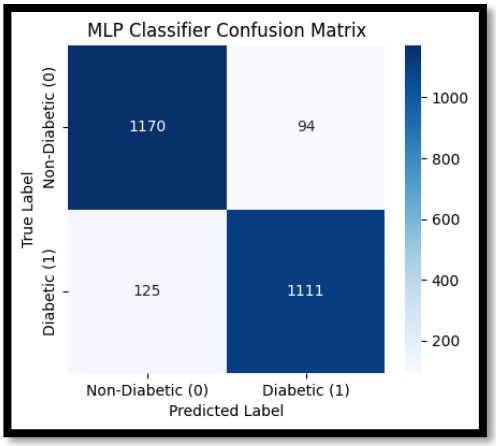


Fig. 52. Best MLP model confusion matrix (experiment 57)

Included Hyperparameter in MLPClassifier	Best experiment	Training Accuracy	Overall Accuracy
Hidden Layer/s	3	1	.9016
Neurons per layer	13	1	.9052
Activation Function (ReLU)	22	.9975	.9016
Activation Function (tanh)	29	.9999	.8956
Optimizer (adam)	36	.9276	.91
Optimizer (sgd)	51	.8969	.8988
Learning Rate (adam)	57	.9345	.9124
Learning Rate (sgd)	74	.8968	.8988
Batch Size	94	.9381	.9088
Epoch	107	.9345	.9124

Table 1. Hyperparameter effects Summarization

i) Analysis of Strengths and Weaknesses of Each Model

The experimental results revealed that various hyperparameters affected the models' generalization and stability in different manners. Adding depth to hidden layers initially resulted in improvements but soon resulted in diminishing returns. Neurons per layer also exhibited a similar trend: small increases resulted in slight performance improvement, but after a certain point, increased complexity harmed generalization. Activation functions showed ReLU to generally be more accurate than Tanh in absolute testing accuracy, although both suffered from overfitting risks with or near-perfect training accuracies. When optimizing, Adam consistently produced higher testing accuracies than SGD, although SGD provided tighter generalization at slightly lower performance potential. Learning rate experiments verified that an optimally tuned learning rate has a substantial effect on performance, particularly when paired with the appropriate optimizer. Lower rates were typically safer, as aggressive rates such as 1.0 greatly hurt testing accuracy. Finally, experiments with batch size showed that while a batch size of 128 did a little better than 32 in testing accuracy without regularization, the smaller batch size provided a more balanced performance in both training and testing, avoiding the slight underfitting that occurred with larger batch sizes.

Throughout every aspect, the most prominent strength of top-performing models was excellent generalization without gross overfitting, whereas the weakness was largely confined to either over-complexity or underfitting where hyperparameters were over-extended.

j) Hyperparameter impact discussion

Throughout all experiments, the variation in the solver, i.e., changing to Adam or SGD, affected model performance most significantly. The choice of optimizer directly impacted both the speed at which the model converged and the model's ability to generalize. Adam models always had the highest test accuracies, well above the 91% level, in experiments such as Experiment 57, whereas SGD models, while having slightly lower maximum test accuracies of about 89.88%, exhibited highly stable and closely matched training and testing outcomes, showing good generalization. By comparison, models that employed the default lbfgs solver earlier in the experiments found it difficult to consistently break the 90% ceiling and tended to exhibit signs of overfitting with increased complexity. These observations implies that the optimizer is not only crucial in improving raw performance but also in striking a balance between the trade-off among learning speed, overfitting, and generalization. While other hyperparameters such as hidden layers, neurons, and batch size had significant impacts, none altered the general model behavior and prediction abilities as drastically as the choice of solver did.

V. Conclusion and Reflection

a) Key findings

The experiments conducted in this laboratory exercise confirmed the significant role that hyperparameter choice plays on neural network model performance and generalization. Notable findings were that adding hidden layers and neurons beyond an optimum resulted in diminishing returns, while activation functions such as ReLU performed better compared to Tanh on plain testing accuracy. Optimizer selection was the most significant hyperparameter, with Adam producing the best test accuracies but SGD having narrower generalization margins. Tuning the learning rate had a significant influence on convergence and performance, where conservative settings prevented instability, and batch size experiments showed a trade-off between computation speed and risk of both overfitting and underfitting. Generally, models that optimally balanced complexity and regularization performed the best without overfitting.

b) Experience working with real-world data and applying the Neural Network

This lab exercise's real-world data presented both difficulties and benefits. The need to manage different levels of model complexity, hyperparameter interactions, and stability issues provided a genuine view into practical machine learning applications outside of textbook illustrations. Using neural networks on actual data underlined the need of methodical experimentation and rigorous assessment, hence supporting the idea that theoretical knowledge by itself is inadequate without practical trial and error.

c) Challenges encountered during the exercise

Many difficulties arose during the activity. Often, early trials yielded training accuracies near 1.0 because the models were overparameterized in relation to the quantity and complexity of the dataset. Without suitable regularization, high model capacity allowed the neural network to remember training data instead of learn general patterns, which resulted in bad generalization on unknown data. Tuning optimizers and learning rates presented another difficulty; aggressive settings sometimes compromised performance. A more organized hyperparameter tuning approach, such as using grid search or Bayesian optimization, would be recommended to address these difficulties in future predictive modeling projects. Including regularization techniques earlier in the model construction process also helps to regulate overfitting when testing with more powerful networks.

VI. References

[1] "Multilayer Perceptrons in Machine Learning," DataCamp, [Online]. Available: https://www.datacamp.com/tutorial/multilayer-perceptrons-in-machine-learning. [Accessed: 27-Oct-2023].

[2] M. Mustafa, “Diabetes Prediction Dataset,” Kaggle, 2021. [Online]. Available: https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset

[3] American Diabetes Association, “2. Classification and Diagnosis of Diabetes: Standards of Medical Care in Diabetes—2023,” Diabetes Care, vol. 46, no. Supplement 1, pp. S19–S40, Jan. 2023.

[4] P. L. Jimeno et al., “Philippine Practice Guidelines on the Diagnosis and Management of Diabetes Mellitus,” Journal of the ASEAN Federation of Endocrine Societies, vol. 37, no. 1, 2022.

[5] World Health Organization, “Obesity and overweight,” WHO Fact Sheet, Jun. 2021. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/obesity-and-overweight.

[6] GeeksforGeeks, "How to split a Dataset into Train and Test Sets using Python," Apr. 18, 2025. [Online]. Available: https://www.geeksforgeeks.org/how-to-split-a-dataset-into-train-and-test-sets-using-python/. [Accessed: Apr. 25, 2025].

[7] V. Dasari, "Multilayer Perceptron and its Hyperparameter Tuning," Medium, Nov. 7, 2024. [Online]. Available: https://medium.com/@vijaikdasari/multilayer-perceptron-and-its-hyperparameter-tuning-deedfbca2d1b. [Accessed: Apr. 25, 2025].